Estoque Inteligente

Manual de montagem e operação

Uma estação de leitura para identificar as seis caixas e manter o estoque registrado no modelo 3D. Projeto baseado nas fotos e na documentação fornecidas pelo grupo.



Antes de começar

O sistema utiliza um ESP32 clássico WROOM/DevKit. As tags ficam dentro das caixas. O operador escolhe a vaga e confirma a operação aproximando a caixa da estação.

Limite importante: um leitor RFID e um ultrassônico não localizam automaticamente seis caixas nas prateleiras. O 3D mostra os registros feitos pelo fluxo, não uma medição independente de cada vaga.

Código e documentação completa: github.com/DanielDalPra/Mundo-Senai-IOT

Peças e estação física

Componente	Uso
ESP32 WROOM/DevKit	Wi-Fi, sensores, leitor e display. A variante deve expor os GPIOs da tabela.
RC522 + seis tags distintas	Identificação da caixa por UID. Tags compatíveis de 13,56 MHz.
HC-SR04	Presença da caixa na estação de leitura.
TCS3200 / Color Sensor V443	Cor complementar; não identifica a caixa.
OLED SPI de seis pinos	Mensagens de operação e conexão. SSD1306 128×64 assumido conforme o DOCX.
Protoboard, jumpers e USB de dados	Montagem temporária, gravação e alimentação.
Resistores 2,2 kOhm + 3,3 kOhm	Divisor do ECHO. Preferir 1%; não identificados nas fotos.

Reposicione para ler a mesma caixa

Os sensores aparecem em lados diferentes nas fotos. Monte uma estação à frente do depósito: o RFID fica perto da etiqueta, o ultrassônico aponta para a face da mesma caixa e o sensor de cor observa uma área colorida uniforme.

1. RFID junto à etiqueta
Fixe o RC522 sob papelão fino ou ao lado da caixa. Oriente a tag interna paralela à antena e teste o alcance real. Evite metal e duas tags próximas ao mesmo tempo.

2. Batente para o ultrassônico
Comece posicionando a face da caixa a cerca de 3 cm. O limite de entrada é 4 cm e o de liberação é 6 cm; esses valores precisam ser testados.

3. Cor com luz estável
Comece a aproximadamente 1-2 cm de uma área colorida, ajuste conforme o módulo e proteja de variações de luz. Mantenha o OLED visível.

4. Marque as prateleiras
A1, A2, A3 na superior; B1, B2, B3 na inferior, da esquerda para a direita quando vistas de frente.

Pinagem revisada

Números da tabela são GPIO. Use os rótulos impressos, não a posição dos pinos. **Não misture esta pinagem com a tabela do documento antigo.**

Módulo	Terminal	Destino / ligação
OLED	VCC / GND	3V3 / GND
OLED	CLK (D0)	GPIO18, compartilhado
OLED	MOSI (D1)	GPIO23, compartilhado
OLED	CS / D/C	GPIO16 / GPIO27
OLED	RESET	Sem terminal na foto; configuração -1
RC522	3.3V / GND	3V3 / GND
RC522	SCK / MOSI / MISO	GPIO18 / GPIO23 / GPIO19
RC522	SDA (SS) / RST	GPIO21 / GPIO22
RC522	IRQ	Sem conexão
HC-SR04	VCC / GND	5 V / GND
HC-SR04	TRIG	GPIO25
HC-SR04	ECHO	Divisor 2,2 kOhm + 3,3 kOhm, depois GPIO26
TCS3200	VCC / GND	3V3 / GND
TCS3200	S0 / S1	GPIO32 / GPIO33
TCS3200	S2 / S3	GPIO13 / GPIO14
TCS3200	OUT	GPIO34, somente entrada
TCS3200	LED	3V3; confirme a habilitação na bancada

Notas de ligação

O OLED e o RC522 compartilham CLK e MOSI, mas possuem CS separados. SDA/SS do RC522 é seleção SPI neste projeto; não é I²C. Use fios curtos nos primeiros testes.

O módulo de cor fotografado repete VCC/GND nos cabeçalhos. Use a mesma alimentação de 3,3 V. Ele expõe LED, não OE. O GPIO34 não possui saída; está reservado a OUT.

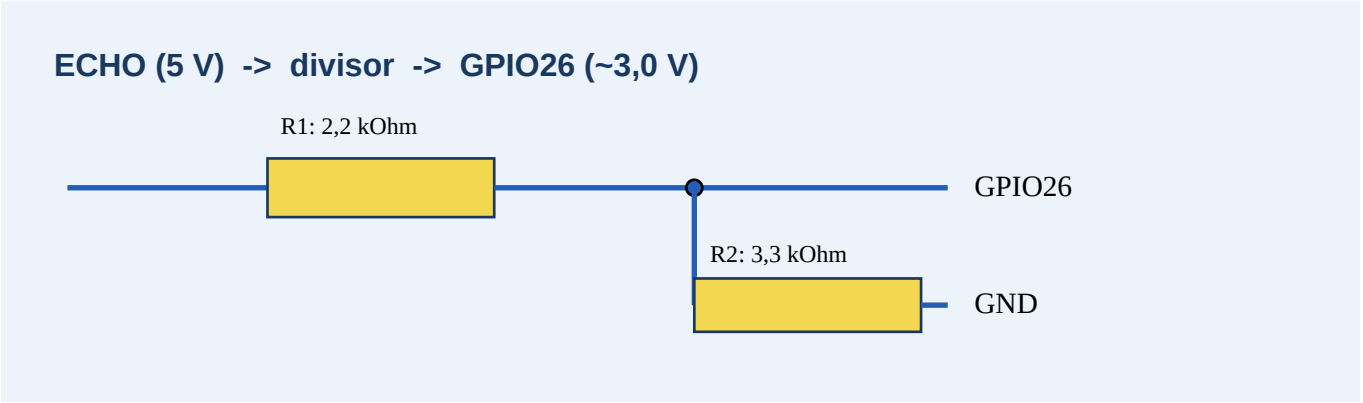
A placa OLED mostra seis terminais: VCC, GND, CLK, MOSI, CS e D/C. Se o módulo real usar outro controlador ou tiver RESET exposto, revise a configuração antes de energizar.

Alimentação e montagem segura

Desconecte o USB antes de mexer nos fios. Todos os módulos devem compartilhar GND.

Alimentação	Ligação
3V3	RC522, OLED e TCS3200. Nunca 5 V no RC522 ou nos GPIOs.
5 V	Somente HC-SR04 neste circuito. Use a linha USB/VIN se a DevKit confirmar 5 V nesse pino; caso contrário, fonte regulada com GND comum.
ESP32	Alimentado por USB. Evite ligar fontes em paralelo sem conhecer o circuito da placa.

Divisor do ECHO: obrigatório



A junção R1/R2 vai ao GPIO26. R1 fica entre ECHO e a junção; R2 entre a junção e GND. Tensão nominal: 5 × 3,3 / (2,2 + 3,3) = 3,0 V. Não ligue o ECHO diretamente ao ESP32.

1. Confira os trilhos
Verifique continuidade da protoboard, GND comum e separação entre 3V3 e 5 V. Não conecte 5 V no trilho de 3V3.

2. Monte por etapas
Comece com ESP32 e RFID; adicione OLED, depois ultrassônico com divisor, depois o sensor de cor. Confira a tabela antes de cada energização.

3. Teste os LEDs do sensor de cor
O projeto alimenta o TCS3200 em 3,3 V. Se os LEDs ficarem fracos, use luz externa estável e calibre. Mudar para 5 V exige adaptação dos sinais lógicos.

4. Fixe após testar
Só fixe sensores e organize os fios quando Wi-Fi, leitura das tags e presença estiverem confiáveis na posição final.

Servidor e gravação do ESP32

No computador: Node.js 24

```
git clone https://github.com/DanielDalPra/Mundo-Senai-IOT.git
cd Mundo-Senai-IOT
npm run setup
npm start
```

Abra **http://localhost:3000**. O comando setup cria o arquivo .env com chaves aleatórias. Use ADMIN_TOKEN para entrar no painel. DEVICE_TOKEN é a chave que será copiada para o ESP32.

Rede local

Mantenha o computador e a placa numa rede com comunicação entre clientes. O ESP32 clássico usa Wi-Fi de 2,4 GHz. No Windows, execute ipconfig e encontre o IPv4 da rede correta. Teste o painel em outro aparelho usando esse IP.

Permita Node.js/porta TCP 3000 no firewall da rede privada autorizada. Redes convidadas podem bloquear comunicação. Não abra a porta no roteador para a internet.

Firmware: PlatformIO no VS Code

1. Abra a pasta firmware

Instale a extensão PlatformIO IDE. O ambiente esp32dev e as bibliotecas estão definidos em platformio.ini.

2. Crie a configuração local

Copie include/config.example.h para include/config.h. Preencha os campos abaixo.

```
#define WIFI_SSID "nome-da-rede"
#define WIFI_PASSWORD "senha-da-rede"
#define SERVER_URL "http://192.168.1.100:3000"
#define DEVICE_TOKEN "valor-do-DEVICE_TOKEN-no-env"
#define DEVICE_ID "esp32-maquete-01"
```

Troque os exemplos pelos seus valores. SERVER_URL não usa localhost, nem /api, nem barra final. Nunca use ADMIN_TOKEN no firmware. Os arquivos .env e config.h são ignorados pelo Git.

3. Compile, grave e monitore

Clique em Build e Upload. Use cabo USB de dados. Abra o monitor serial a 115200 baud. Se necessário, confira a porta COM e segure BOOT no início da gravação.

A compilação sem config.h usa exemplos para permitir validação do código; não conectará à sua rede até a configuração real ser preenchida.

Cadastro e operação das caixas

Cadastre as seis tags

1. Verifique a conexão
Inicie o servidor e ligue a placa. O painel deve mostrar ESP32 conectado. Confirme presença e distância colocando uma caixa na estação.

2. Leia e vincule
Na aba Tags, apresente uma caixa, use a última leitura e vincule o UID à CX correspondente. Não use o UID de exemplo como se fosse uma tag real.

3. Retire entre leituras
Retire a caixa por cerca de um segundo para liberar a estação. Repita com as seis caixas. Cada tag só pode pertencer a uma caixa.

Faça um movimento

Ação	Sequência
Entrada	Selecione caixa fora do estoque + vaga livre. Prepare a leitura.
Saída	Retire a caixa da vaga e selecione saída para essa caixa.
Mudar de vaga	Selecione caixa já registrada e uma nova vaga livre.

Após preparar, **espere a operação aparecer no OLED** e então aproxime a caixa. A leitura da tag esperada confirma o registro. O prazo é 60 segundos. Depois da entrada ou mudança, posicione fisicamente a caixa na vaga escolhida.

O que o 3D representa

O modo conectado inicia sem ocupação presumida, mesmo que as fotos mostrem caixas na estante. O servidor é a fonte das posições. A demonstração manual é separada e não altera esse estoque.

Se a conexão cair

O ESP32 conserva uma leitura pendente e a reenvia com o mesmo ID. Se o servidor já salvou o movimento, ele não repete a operação. Uma leitura com comando expirado é recusada; prepare novamente e reapresente a caixa.

Sem comunicação, não continue movimentando várias caixas esperando uma fila automática: há apenas uma leitura pendente. Se alguém mover uma caixa fora do fluxo, use a correção manual com motivo e confira a maquete.

Preserve os registros

O servidor guarda estado e histórico em data/state.json. Para backup, pare o servidor e copie esse arquivo. Reiniciar preserva as caixas, mas cancela a operação pendente.

Calibração e diagnóstico

Presença

Entrada: duas leituras abaixo de 4 cm. Liberação: três leituras acima de 6 cm ou sem eco. Ajuste PRESENCE_ENTER_CM e PRESENCE_EXIT_CM no config.h após testar o batente. Um timeout é enviado como distância nula, nunca como zero.

Cor

pulseIn mede duração em microssegundos. O firmware usa ganho/pulso, e trata zero como falha. Para calibrar, coloque um cartão branco na posição final, anote a mediana de dez leituras R/G/B e use essas durações em COLOR_GAIN_R, COLOR_GAIN_G e COLOR_GAIN_B. Grave novamente e teste as caixas.

A cor é complementar: várias caixas têm a mesma cor. UID é a identidade. Se a iluminação não estiver estável, aceite indefinida ou desative COLOR_ENABLED até ajustar o módulo.

Sintoma	O que conferir
RC522 0x00 / 0xFF	3V3, GND, CS21, RST22, MISO19, MOSI23 e SCK18.
Tag não aparece	Presença na estação, alcance da etiqueta interna, orientação, metal e compatibilidade da tag.
Só lê uma vez	Retire a caixa por cerca de um segundo e reapresente.
OLED apagado	SPI de seis pinos, alimentação, CS16, DC27 e controlador/resolução reais.
Sem eco	5 V no HC-SR04, divisor correto, alvo plano e GND comum.
Wi-Fi sem servidor	IP/porta, firewall, isolamento de clientes e SERVER_URL.
HTTP 401 / 403	DEVICE_TOKEN e DEVICE_ID iguais aos valores do servidor.
Cor indefinida	OUT34, S0/S1, filtros, pulsos zero, distância e luz.
3D difere da maquete	Movimento sem leitura: corrigir posição com motivo.

Teste pelo menos dez inserções/retiradas por caixa, incluindo leitura da tag errada e tentativa de vaga ocupada. Faça isso na montagem final e na iluminação da apresentação.

Validação e referências

Validação de software

A suíte npm test verifica cadastro e UID único, entrada, saída e mudança de vaga, conflitos, prazo, cancelamento, leitura errada, repetição de evento, persistência, autenticação, SSE e proteção de rotas. O teste de navegador também verifica login, cadastro, confirmação RFID simulada, sincronização do 3D e layout móvel.

O registro atualizado da compilação e dos testes está em **docs/TESTES.md**. Testes simulados validam o software, não o circuito físico. Não foi conectado nem gravado o ESP32 do grupo nesta entrega.

Teste sem placa

```
npm test
npm run simulate -- --watch
# Em outro terminal, exemplo de leitura:
npm run simulate -- --uid 04A1B201
```

Cadastre o UID fictício e prepare uma operação antes de repetir a leitura. Pare o simulador antes de usar o ESP32 real. Os dados simulados usam a mesma API e devem ser reconciliados antes da apresentação.

Documentação completa no repositório

docs/MONTAGEM.md: pinagem, componentes e alimentação.
docs/INSTALACAO.md: servidor, Wi-Fi, upload e recuperação.
docs/CALIBRACAO.md: sensores e diagnóstico.
docs/ARQUITETURA-API.md: endpoints, protocolo e modelo.
docs/TESTES.md: evidências e roteiro de bancada.

Fontes técnicas

Espressif: ESP32, GPIO e limites elétricos

NXP: MFRC522

Adafruit: HC-SR04 e ECHO de 5 V

ams OSRAM: TCS3200

Adafruit: biblioteca SSD1306

Os anexos do grupo foram tratados como referência técnica. O DOCX original, as fotos e o vídeo estão preservados em referencias/. A pinagem foi revisada; as instruções deste manual correspondem ao firmware do repositório.